BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi web yang sangat pesat menyebabkan halaman web modern memiliki struktur yang semakin kompleks. Setiap halaman web dibangun menggunakan *Hypertext Markup Language* (HTML) yang secara umum dapat direpresentasikan dalam bentuk struktur hierarkis yang dikenal sebagai *Document Object Model* (DOM). Struktur DOM ini berbentuk pohon (*tree*) yang terdiri dari elemen-elemen HTML yang saling terhubung melalui hubungan induk (*parent*) dan anak (*child*).

Meskipun struktur DOM merupakan konsep dasar dalam pengembangan web, representasinya dalam bentuk kode HTML sering kali sulit dipahami secara langsung, terutama pada halaman web yang memiliki banyak elemen. Oleh karena itu, diperlukan suatu cara yang dapat membantu menyajikan struktur tersebut dalam bentuk yang lebih mudah dipahami.

Salah satu cara yang dapat digunakan adalah dengan memvisualisasikan struktur halaman web dalam bentuk graf. Dalam pendekatan ini, setiap elemen HTML direpresentasikan sebagai simpul (*node*) dan hubungan antar elemen direpresentasikan sebagai sisi (*edge*). Dengan demikian, struktur DOM dapat dilihat secara lebih jelas dan terstruktur.

Berdasarkan permasalahan tersebut, tugas akhir ini berfokus pada pengembangan perangkat lunak yang mampu mengambil dokumen HTML dari suatu *Uniform Resource Locator* (URL), membangun struktur DOM, dan merepresentasikannya dalam bentuk graf yang dapat divisualisasikan. Perangkat lunak ini diharapkan dapat membantu pengguna dalam memahami struktur halaman web secara lebih mudah.

1.2 Rumusan Masalah

Rumusan Masalah yang akan dibahas pada tugas akhir ini adalah:

1. Bagaimana cara mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM?
2. Bagaimana cara merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis?
3. Bagaimana merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna?

1.3 Tujuan

Tujuan yang ingin dicapai skripsi ini adalah sebagai berikut:

1. Mengambil dokumen HTML dari suatu situs web berdasarkan URL serta membangun struktur DOM.
2. Merepresentasikan struktur DOM ke dalam bentuk graf yang terdiri dari (*node*) dan (*edge*) melalui proses traversal yang sistematis.

3. Merancang perangkat lunak yang mampu menghasilkan visualisasi graf dari struktur halaman web secara otomatis dan mudah dipahami pengguna.

1.4 Batasan Masalah

Pada pengerjaan tugas akhir ini terhadap batasan sebagai berikut:

- Perangkat lunak dikembangkan menggunakan bahasa pemrograman Java

1.5 Metodologi

Metodologi pengerjaan tugas akhir ini adalah sebagai berikut:

1. Melakukan studi literatur mengenai HTML, DOM, struktur data graf, Graphviz, dan metode traversal.
2. Melakukan analisis kebutuhan serta perancangan arsitektur perangkat lunak yang akan dikembangkan.
3. Mengimplementasikan sistem untuk mengambil dokumen HTML, membangun struktur DOM, serta merepresentasikannya dalam bentuk graf.
4. Melakukan pengujian dan eksperimen menggunakan beberapa halaman web dengan struktur yang berbeda.
5. Melakukan analisis terhadap hasil visualisasi yang dihasilkan oleh sistem.
6. Menyusun dokumentasi dan laporan tugas akhir.

1.6 Sistematika Pembahasan

Sistematika pembahasan skripsi ini adalah sebagai berikut:

- **Bab 1:** Pendahuluan
  Membahas latar belakang, rumusan masalah, tujuan, batasan masalah, metodologi, dan sistematika pembahasan.
- **Bab 2:** Landasan Teori
- Bab 3: Analisis
- Bab 4: Perancangan
- Bab 5: Implementasi dan Pengujian
- Bab 6: Kesimpulan dan Saran

BAB 2

LANDASAN TEORI

2.1 Hypertext Markup Language (HTML)

2.1.1 Pengertian HTML

Hypertext Markup Language (HTML) merupakan bahasa markup yang digunakan untuk menyusun struktur dasar halaman web. HTML tidak termasuk bahasa pemrograman, melainkan bahasa yang digunakan untuk mendeskripsikan bagaimana konten ditampilkan pada browser. HTML memungkinkan pengembang untuk mengatur berbagai jenis konten seperti teks, gambar, tautan, tabel, dan elemen multimedia lainnya.

Menurut *HTML and CSS: Design and Build Websites*, HTML berfungsi sebagai kerangka utama dari halaman web, yang menentukan bagaimana setiap bagian konten disusun sehingga dapat ditampilkan secara terstruktur oleh browser.

HTML bekerja dengan konsep *tag-based structure*, di mana setiap elemen ditandai dengan tag tertentu. Dengan adanya struktur ini, browser dapat memahami peran masing-masing elemen dan menampilkannya dengan benar.

2.1.2 Elemen dan Tag HTML

Elemen HTML merupakan komponen dasar dalam penyusunan halaman web. Setiap elemen biasanya terdiri dari tag pembuka dan tag penutup, serta isi di antara keduanya. Sebagai contoh, elemen paragraf dituliskan dengan sebagai tag pembuka dan sebagai tag penutup.

Selain itu, elemen HTML juga dapat memiliki atribut yang memberikan informasi tambahan. Atribut ini dituliskan di dalam tag pembuka, seperti atribut href pada elemen <a> untuk menentukan alamat tautan.

Struktur elemen yang saling bersarang (*nested elements*) memungkinkan terbentuknya hubungan hierarkis antar elemen. Hal ini menjadi dasar penting dalam pembentukan struktur DOM.

```html
<!DOCTYPE html>
<html lang="id">
<head>
    <meta charset="UTF-8">
    <title>Judul Halaman</title>
</head>
<body>
    <h1>Ini Judul Utama</h1>
    <p>Ini adalah paragraf konten.</p>
</body>
</html>
```

Gambar 2.1: Contoh struktur dasar dokumen HTML

2.2 Struktur Dokumen HTML

2.2.1 Struktur dasar HTML

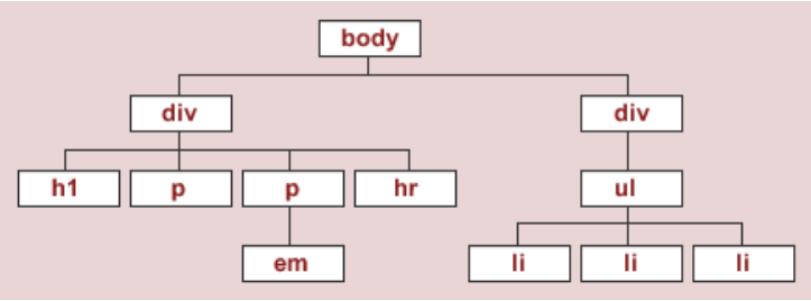
Dokumen HTML memiliki struktur standar yang terdiri dari beberapa bagian utama, yaitu <html>, <head>, dan <body>. Elemen <html> merupakan root dari seluruh dokumen. Elemen <head> berisi informasi metadata seperti judul halaman, sedangkan <body> berisi konten utama yang ditampilkan kepada pengguna.

Struktur ini memastikan bahwa setiap halaman web memiliki susunan yang konsisten dan dapat dipahami oleh browser.

2.2.2 Hierarki elemen HTML

Struktur HTML bersifat hierarkis, yang berarti elemen dapat berada di dalam elemen lain. Misalnya, elemen dapat berisi elemen <p>, <img>, atau elemen lainnya. Hubungan ini membentuk struktur bertingkat yang menyerupai pohon.

Hierarki ini penting karena menentukan bagaimana elemen saling berhubungan dan bagaimana struktur tersebut akan diproses menjadi DOM. Semakin kompleks halaman web, maka semakin dalam dan luas struktur hierarki yang terbentuk.



Gambar 2.2: Representasi hierarki elemen HTML dalam bentuk pohon

2.3 Document Object Model (DOM)

2.3.1 Pengertian DOM

Document Object Model (DOM) adalah representasi dari dokumen HTML dalam bentuk struktur objek yang dapat diakses dan dimanipulasi oleh program. DOM memungkinkan setiap elemen dalam halaman web diperlakukan sebagai objek yang memiliki properti dan metode.

Menurut *World Wide Web Consortium*, DOM menyediakan antarmuka standar untuk mengakses dan memodifikasi dokumen HTML.

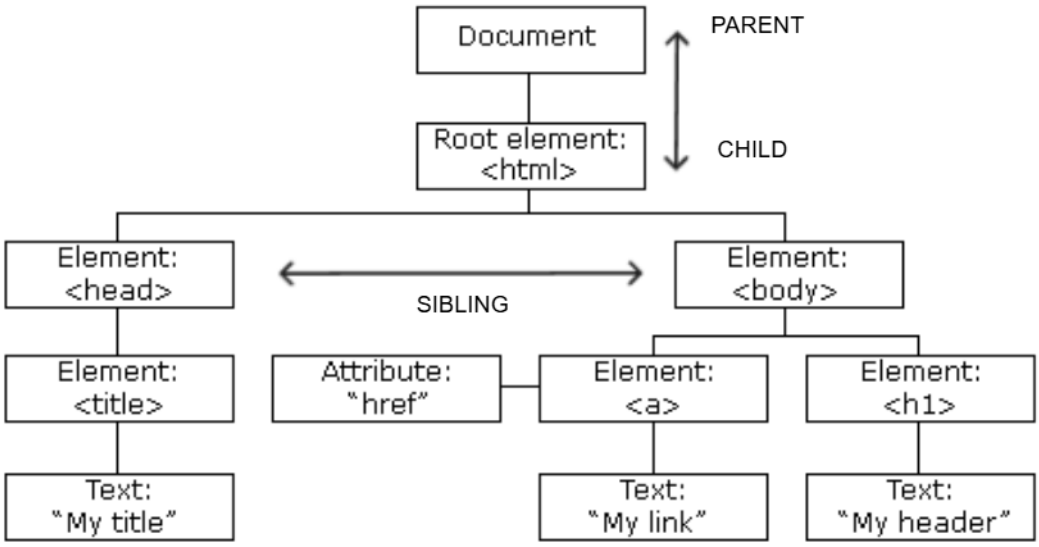
Dengan adanya DOM, halaman web tidak hanya menjadi tampilan statis, tetapi juga dapat diproses dan dimodifikasi secara dinamis.

2.3.2 Struktur DOM sebagai Tree

Struktur DOM berbentuk pohon (tree structure), di mana setiap elemen HTML direpresentasikan sebagai node. Node paling atas disebut root node, yang biasanya adalah elemen <html>. Setiap node dapat memiliki:

- Parent (induk)
- Child (anak)
- Sibling (saudara)

Struktur ini memungkinkan proses traversal dilakukan secara sistematis, seperti menggunakan DFS.



Gambar 2.3: Struktur DOM dalam bentuk Tree

2.3.3 Jenis node pada DOM

DOM terdiri dari beberapa jenis node, antara lain:

- Element node: merepresentasikan tag HTML
- Text node: merepresentasikan isi teks
- Attribute node: merepresentasikan atribut

Keberadaan berbagai jenis node ini membuat struktur DOM lebih kompleks dibandingkan HTML mentah, sehingga analisis DOM memberikan informasi yang lebih lengkap.

2.4 Struktur Data Graf

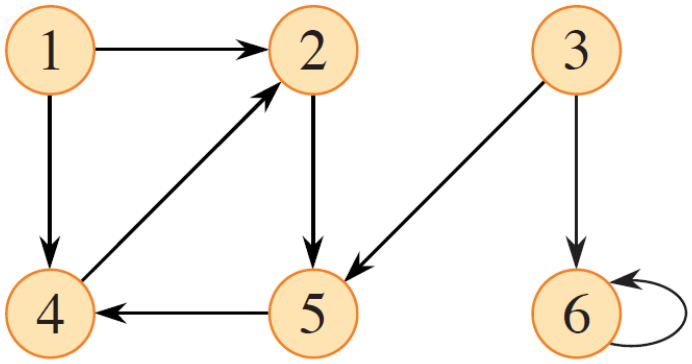
2.4.1 Pengertian Graf

Graf adalah struktur data yang terdiri dari simpul (node) dan sisi (edge) yang menghubungkan simpul tersebut.

Menurut *Introduction to Algorithms*, graf digunakan untuk merepresentasikan hubungan antar objek dalam berbagai bidang komputasi.

2.4.2 Graf Berarah

Graf berarah adalah graf yang memiliki arah pada setiap edge. Dalam konteks DOM, hubungan parent ke child memiliki arah, sehingga dapat dimodelkan sebagai graf berarah.



Gambar 2.4: Contoh graf berarah sederhana (node dan edge)

2.4.3 Representasi DOM sebagai graf

Struktur DOM dapat direpresentasikan sebagai graf dengan cara memetakan setiap elemen HTML sebagai node dan hubungan parent-child sebagai edge.

Pendekatan ini memungkinkan struktur halaman web dianalisis menggunakan teori graf, serta mempermudah visualisasi menggunakan perangkat lunak seperti Graphviz.

2.5 Traversal Graph

2.5.1 Pengertian Traversal

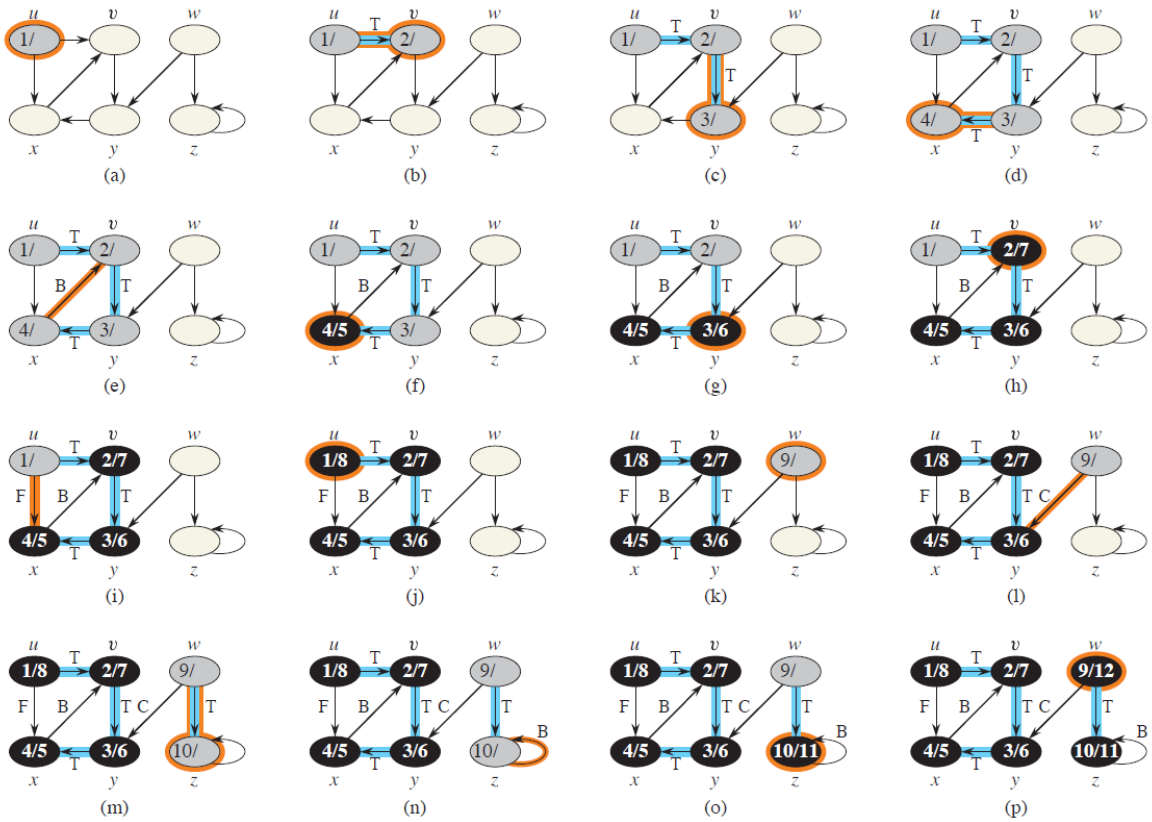
Traversal graf adalah proses mengunjungi setiap node dalam graf secara sistematis. Traversal diperlukan untuk memastikan seluruh struktur graf dapat diproses tanpa ada node yang terlewat.

2.5.2 Depth-First Search

Depth-First Search (DFS) adalah metode traversal yang bekerja dengan menelusuri node hingga ke bagian terdalam sebelum kembali ke node sebelumnya.

Metode ini sangat cocok untuk struktur berbentuk tree seperti DOM karena mengikuti pola parent-child secara mendalam.

Menurut *Introduction to Algorithms*, DFS sering digunakan untuk eksplorasi struktur hierarkis dan graf.



Gambar 2.5: Ilustrasi proses DFS pada graf berarah

2.6 Graphviz

2.6.1 Pengertian Graphviz

Graphviz adalah perangkat lunak yang digunakan untuk memvisualisasikan graf berdasarkan deskripsi teks.

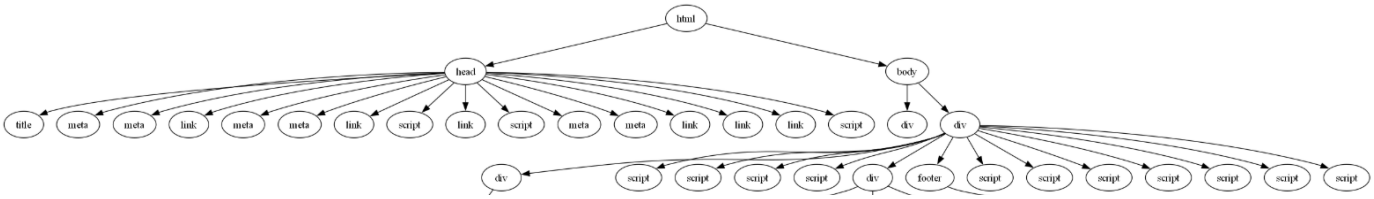
Menurut 2Graphviz Project, Graphviz menyediakan berbagai algoritma layout untuk menampilkan graf secara otomatis.

2.6.2 Format DOT

Graphviz menggunakan format DOT untuk mendefinisikan graf.

```
digraph DOM {
node [shape=oval];
node1 [label="html"];
node2 [label="head"];
node1 -> node2;
node3 [label="link"];
node2 -> node3;
node4 [label="script"];
node2 -> node4;
node5 [label="script"];
node2 -> node5;
node6 [label="script"];
```

Gambar 2.6: Potongan Kode dengan format DOT



Gambar 2.7: Visualisasi dari format DOT dengan menggunakan Graphviz

%endtabular

2.6.3 Kutipan

- Buku: [1]
- Buku: [2]

DAFTAR REFERENSI

[1] Duckket, J. (2011) *HTML and CSS, Design and Build Websites*. John Wiley and Sons, Inc, Canada.

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., dan Stein, C. (2022) *Introduction to Algorithms*, 4th edition. The MIT Press, America.

LAMPIRAN A

KODE PROGRAM

Kode A.1: MyCode.c

```c

// This does not make algorithmic sense,
// but it shows off significant programming characters.

#include<stdio.h>

void myFunction( int input, float* output ) {
    switch ( array[i] ) {
        case 1: // This is silly code
            if ( a >= 0 || b <= 3 && c != x )
                *output += 0.005 + 20050;
            char = 'g';
            b = 2^n + ~right_size - leftSize * MAX_SIZE;
            c = (--aaa + &daa) / (bbb++ - ccc % 2 );
            strcpy(a,"hello $@?");
    }
    count = ~mask | 0x00FF00AA;
}

// Fonts for Displaying Program Code in LATEX
// Adrian P. Robson, nepsweb.co.uk
// 8 October 2012
// http://nepsweb.co.uk/docs/progfonts.pdf
```

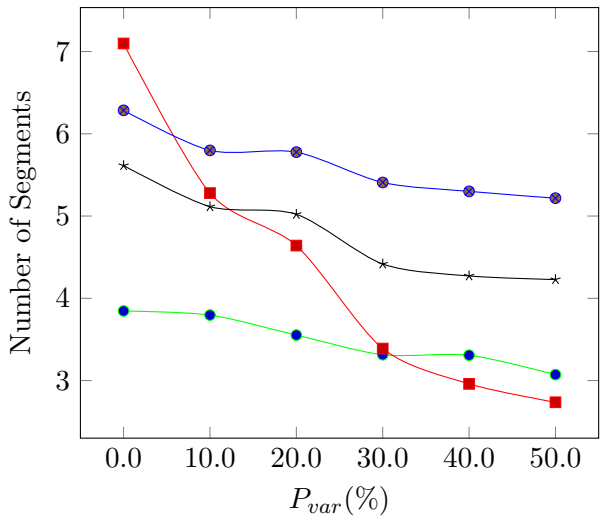
Kode A.2: MyCode.java

```java
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashSet;

//class for set of vertices close to furthest edge
public class MyFurSet {
    protected int id;                                   //id of the set
    protected MyEdge FurthestEdge;                      //the furthest edge
    protected HashSet<MyVertex> set;                    //set of vertices close to furthest edge
    protected ArrayList<ArrayList<Integer>> ordered;    //list of all vertices in the set for each trajectory
    protected ArrayList<Integer> closeID;               //store the ID of all vertices
    protected ArrayList<Double> closeDist;              //store the distance of all vertices
    protected int totaltrj;                             //total trajectories in the set

    /*
     * Constructor
     * @param id : id of the set
     * @param totaltrj : total number of trajectories in the set
     * @param FurthestEdge : the furthest edge
     */
    public MyFurSet(int id,int totaltrj,MyEdge FurthestEdge) {
        this.id = id;
        this.totaltrj = totaltrj;
        this.FurthestEdge = FurthestEdge;
        set = new HashSet<MyVertex>();
        ordered = new ArrayList<ArrayList<Integer>>();
        for (int i=0;i<totaltrj;i++) ordered.add(new ArrayList<Integer>());
        closeID = new ArrayList<Integer>(totaltrj);
        closeDist = new ArrayList<Double>(totaltrj);
        for (int i = 0;i <totaltrj;i++) {
            closeID.add(-1);
            closeDist.add(Double.MAX_VALUE);
        }
    }

}
```

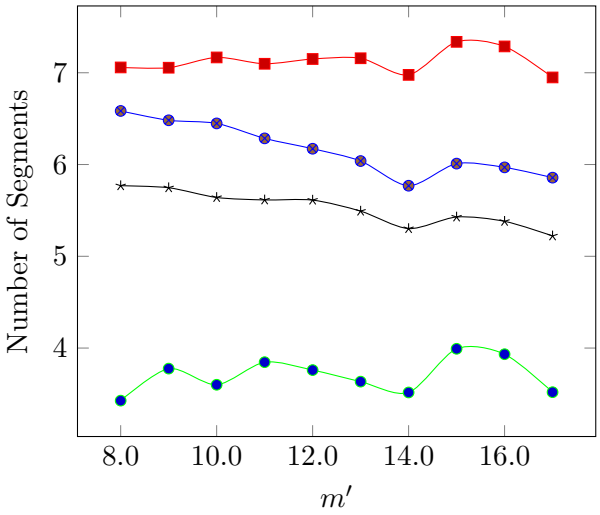
LAMPIRAN B

HASIL EKSPERIMEN

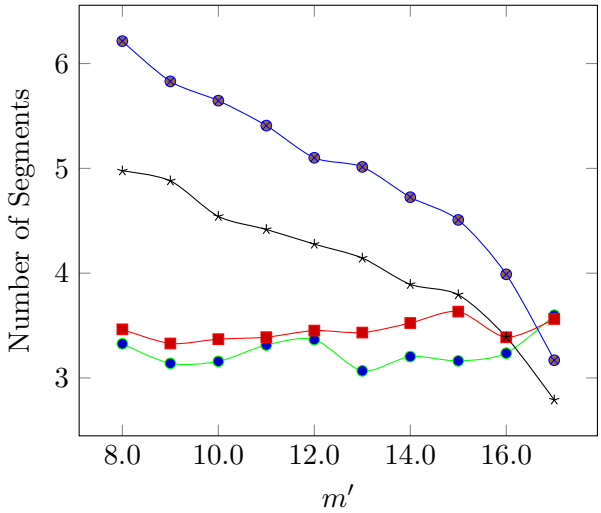
Hasil eksperimen berikut dibuat dengan menggunakan TIKZPICTURE (bukan hasil excel yg diubah ke file bitmap). Sangat berguna jika ingin menampilkan tabel (yang kuantitasnya sangat banyak) yang datanya dihasilkan dari program komputer.



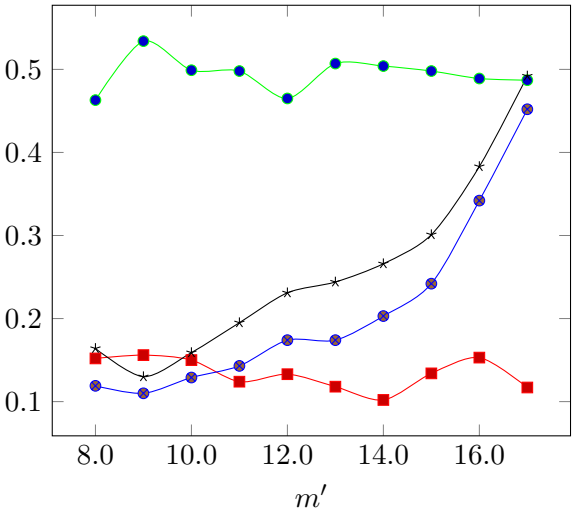
Gambar B.1: Hasil 1



Gambar B.2: Hasil 2



Gambar B.3: Hasil 3



Gambar B.4: Hasil 4